

Plan: Agency Hub — phases 1–6 (runnable mock-mode app)

Context

The user wants a white-label, multi-tenant dashboard for insurance agencies and other small businesses:

- Frontend: Angular 19+.
- Backend: NestJS.
- Data and auth: Firebase (Auth + Firestore).
- Integrations later: n8n.

The previous turn agreed the blueprint. This build delivers phases 1–6 as a runnable app that uses only mock data and Firebase emulators, so the user can test login, RTL/LTR and the dashboard without any external accounts.

User decisions:

- **Location:** a real folder the user picks. At the start of implementation, call ``request_directory`` (or ``change_directory``) and build there, not in the scratch workspace.
- **Scope:** phases 1–6. Firestore rules tests, the full seed and n8n exports are a later round.
- **Languages:** Hebrew (default, RTL) and English (LTR).
- **Repo tooling:** plain npm workspaces, not Nx.

Repo layout

...

agency-hub/

package.json workspaces: web, api, shared; scripts: dev, build, test, lint

CLAUDE.md master context (from the blueprint)

firebase.json .firebaserc firestore.rules firestore.indexes.json

firebase/seed-dev.ts minimal dev seed: 1 tenant, demo users (owner, agent); credentials documented in README only

shared/ @agency-hub/shared — DTOs/interfaces: User, Tenant, Customer, Message, Policy, Lead, DashboardSummary, enums

web/ Angular 19+ app (LESS, standalone)

api/ NestJS 11 app

docs/firestore-model.md

README.md setup, env, demo login, mock vs live

...

- Root ``npm run dev`` uses ``concurrently`` to run the Firebase emulators (Auth + Firestore), ``api`` (watch) and ``web`` (ng serve, with a proxy so ``/api`` → localhost:3000).
- The emulators need Java 11+. Check for it and note it in the README.

Phase 1 — Scaffold

- Generate the apps:

- ``npx @angular/cli new web --style=less --routing --ssr=false --skip-git``
- ``npx @nestjs/cli new api --skip-git``

- Wire up the npm workspaces, the strict root tsconfig base, ESLint and Prettier.
- Add the `shared` package and consume it from both apps through workspace imports.
- API config: `@nestjs/config` with a zod-validated env. Keys: `DATA\_MODE=mock|live`, `FIREBASE\_PROJECT\_ID`, `FIRESTORE\_EMULATOR\_HOST`, `FIREBASE\_AUTH\_EMULATOR\_HOST`, `CORS\_ORIGINS`, `N8N\_SECRET`.
- Write `.env.example`.

Phase 2 — Frontend shell, i18n, RTL, theming

- `core/i18n/language.service.ts`: `ngx-translate` with the HTTP loader (`assets/i18n/he.json`, `en.json`). A signal holds the current language. On change it sets `` and saves the choice to localStorage (wrapped in try/catch) and to the profile.
- `core/theme/theme.service.ts`: loads `assets/tenants/{slug}/theme.config.json` in `provideAppInitializer`, then writes CSS variables (colors, font, radius, logo URL) to `:root`.
 - The slug comes from the subdomain, then a `?tenant=` query in dev, then the default.
 - Ship two sample tenants (`demo-insurance`, `acme-agency`) with different branding.
- `styles/`:
 - `tokens.less` maps LESS variables to `var(--...)`.
 - Also `breakpoints.less` (mobile first: `@sm 576`, `@md 768`, `@lg 1024`, `@xl 1280`), `mixins.less` and `main.less`.
 - Use logical properties only. Add a `.icon-dir` mirror rule for RTL.
- `layout/shell`:
 - The top bar has the logo, language switch and user menu.
 - Mobile gets a bottom tab bar. From `@md` up there is a side-nav, collapsible on tablet and full on desktop, and it mirrors automatically in RTL.
- `Routes` (`app.routes.ts`):
 - `/auth` (guestGuard): lazy `loadChildren`.
 - `/app` (authGuard, under the shell): dashboard, customers placeholder, messages placeholder, settings placeholder.
 - Wildcard redirect.
- All components are standalone, OnPush, use signals and the new control flow, and use `inject()`.

Phase 3 — Auth (Firebase Auth via `@angular/fire`)

- `Setup`:
 - `provideFirebaseApp`, `provideAuth`, and `connectAuthEmulator` when `environment.useEmulators` is set.
 - App Check is wired but disabled in emulator mode.
- `core/auth/auth.service.ts`: a signal for the current user plus the claims from `getIdTokenResult` (tenantId, role). Methods:
 - login (email/password), `loginWithGoogle` (popup), `loginWithUsername` (calls `POST /api/auth/login` → `signInWithCustomToken`)
 - signup (create user → `sendEmailVerification` → `POST /api/users/bootstrap` → force-refresh the token)
 - `forgotPassword`, `confirmReset`, `changePassword` (reauthenticate first), logout

- **Pages** (`features/auth/`):
 - login (email-or-username field, Google button)
 - signup (strong password validator, terms checkbox)
 - forgot-password: always shows the same message, so it can't be used to check whether an email exists
 - `auth-action`: handles `mode=resetPassword`, `verifyEmail` and `oobCode`
 - All pages use reactive forms and translated text.
- **Guards and interceptor**:
 - `core/auth/guards.ts`: functional `authGuard`, `guestGuard` and `roleGuard(roles)`.
 - `core/api/auth.interceptor.ts`: attaches the Bearer ID token (and the App Check token when enabled), and on a 401 refreshes once, then logs out.

Phase 4 — NestJS core

- **firebase/firebase.module.ts**: `firebase-admin` initialized for the emulators or real credentials. Provides `FIRESTORE` and `FIREBASE_AUTH` tokens.
- **common/**:
 - Guards: `FirebaseAuthGuard` (`verifyIdToken(token, true)`), `TenantGuard` (`tenantId` from claims only) and `RolesGuard`.
 - Decorators: `@CurrentUser()`, `@Roles()` and `@Public()`.
 - Also a global exception filter and an audit interceptor (writes `tenants/{tid}/auditLogs` on mutations).
- **main.ts**:
 - Security: helmet, CORS allow-list, global `ValidationPipe` (`{whitelist, forbidNonWhitelisted, transform}`), `@nestjs/throttler` (global, stricter on `/auth/*`).
 - Logging and docs: `nestjs-pino` with redaction of authorization headers, tokens and passwords, and a `/api` prefix with Swagger in dev.
- **Modules**:
 - `auth`: `POST /auth/login` looks up the username, verifies the password, and returns a custom token. It verifies through the Identity Toolkit REST endpoint, which also works with the Auth emulator.
 - `users`: `POST /users/bootstrap` creates `users/{uid}` and `usernames/{u}` transactionally and sets claims. `GET/PATCH /users/me`.
 - `tenants`: `GET /tenants/current`.
- **firebase/seed-dev.ts**: creates the demo tenant and users in the emulators and sets claims. Run with `npm run seed`. Demo credentials are generated there and written to the README, not hard-coded elsewhere.

Phase 5 — Integrations (mock providers) + dashboard API

- **Pattern for each source in `api/src/integrations/{crm,sheets,gmail,whatsapp}/`**:
 - `*.provider.ts`: the interface plus an injection token.
 - `mock-*.provider.ts`: uses `@faker-js/faker` with he and en locales, insurance-flavoured data and a deterministic seed per tenant.
 - `-live.provider.ts`: a typed stub that throws `NotImplementedException`, with TODO notes for HubSpot, the Sheets API, the Gmail API and the WhatsApp Cloud API.

- **integrations.module.ts**: uses `useFactory` to choose Mock or Live by `DATA_MODE`.
- **Extra mock data**: a mock policies provider for renewals.
- **dashboard/dashboard.service.ts**: `GET /dashboard/summary?range=7d|30d|90d` aggregates all providers with `Promise.allSettled`, so one failing source degrades only its own widget. It returns `DashboardSummary` from `shared`:
 - KPIs
 - per-source blocks
 - renewals due
 - a merged activity feed
 - integration health
- **Supporting endpoints**: `GET /customers` and `GET /messages`, both paginated and backed by the mocks.

Phase 6 — Dashboard UI

- **features/dashboard/dashboard.page.ts**: uses `httpResource` or `rxResource` with a range selector.
- **Widgets in `widgets/`**, each a lazy standalone component (`@defer (on viewport)`):
 - kpi-tiles, crm-pipeline (bar chart), sheets-latest (table → cards on mobile)
 - gmail-inbox, whatsapp-requests (SLA badge), renewals-due, activity-timeline, integration-health
- **Widget states**: each widget has skeleton, empty and error states.
- **Charts**: `ngx-echarts`, mirrored for RTL by reversing the axis and placing the legend with `start`.
- **Grid**: CSS grid with 1 column at base, 2 at `@md` and 4 at `@xl`, using the theme tokens throughout.

Verification

1. In the project folder, run `npm install`, then `npm run seed`, then `npm run dev`. This assumes the emulators start on Java 11+.
2. **Backend checks**:
 - `npm test -w api`: unit tests for the guards (valid, revoked and missing token, cross-tenant access denied), the provider factory and the dashboard aggregation (one provider rejecting).
 - `npm test -w web`: tests for LanguageService (dir switch), ThemeService (CSS variables applied) and the guards.
3. **Browser walkthrough** (built-in browser on localhost:4200):
 - Sign up with a new test account, check that the verification email appears in the emulator UI, then log in with the seeded demo user (email and username).
 - Forgot password: follow the reset link from the emulator UI through `/auth/action`.
 - On the dashboard, check that all widgets render.
 - Switch he ↔ en and confirm the layout mirrors.
 - Resize to 375 / 768 / 1440 and check there is no horizontal scroll and the nav changes form.
 - Load `?tenant=acme-agency` and confirm the branding changes.
4. **API security checks with curl**:
 - `/api/dashboard/summary` without a token → 401.

- Rapid `/api/auth/login` calls → 429.
5. Run `npm run lint` and `npm run build` for all workspaces cleanly.

Out of scope (next round)

- Firestore rules unit tests and a full seed (50 customers and more).
- The n8n ingest module with HMAC, the WhatsApp webhook, and n8n workflow exports.
- Live provider implementations.
- MFA, session cookies and deployment (Cloud Run / Firebase Hosting).